

# LAPORAN PRAKTIKUM

## STRATEGI ALGORITMA

### PRAKTIKUM 3: ALGORITMA GREEDY

#### I. DEFINISI

##### 1.1 Algoritma Greedy

**Definisi:** Algoritma Greedy adalah metode yang paling populer untuk memecahkan persoalan optimasi. Prinsipnya adalah "take what you can get now" yaitu pada setiap langkah selalu dipilih keputusan terbaik secara lokal dengan harapan menghasilkan solusi global yang optimal.

**Cara Kerja:** Solusi dibangun langkah per langkah (step by step). Pada setiap langkah, dari banyak pilihan yang tersedia, dipilih satu pilihan terbaik berdasarkan kriteria tertentu tanpa mempertimbangkan konsekuensi di langkah berikutnya.

**Fungsi:** Digunakan untuk menyelesaikan persoalan optimasi minimasi maupun maksimasi, seperti penukaran uang, penjadwalan, Huffman encoding, dan shortest path.

**Kelebihan:** Implementasi mudah, efisien dalam waktu eksekusi, dan menghasilkan solusi optimal pada sistem koin kanonik (seperti sistem mata uang nyata).

**Kekurangan:** Tidak selalu menghasilkan solusi global yang optimal untuk semua kasus. Pada sistem koin non-kanonik, solusi greedy bisa lebih banyak koin dibanding solusi optimal yang didapat melalui dynamic programming.

##### 1.2 Masalah Penukaran Uang (Coin Exchange)

Diberikan nilai uang  $A$  yang ingin ditukar dan himpunan koin  $C = \{d_1, d_2, \dots, d_n\}$ . Tujuannya adalah menemukan himpunan koin  $S$  sehingga total nilai  $S = A$  dengan jumlah koin seminimum mungkin.

**Fungsi objektif:** *Minimisasi*  $F = \sum x_i$  (jumlah koin yang digunakan)

**Kendala:**  $\sum d_i \times x_i = A$  (total nilai koin harus tepat sama dengan A)

**Strategi Greedy:** Pada setiap langkah, pilih koin dengan nilai terbesar dari himpunan koin yang tersisa selama masih tidak melebihi sisa nilai yang perlu ditukar.

## II. PRE TEST

**Pertanyaan:** Jelaskan apa yang Anda ketahui tentang algoritma Greedy!

**Jawaban:**

Algoritma Greedy adalah strategi perancangan algoritma yang membangun solusi secara bertahap dengan selalu memilih pilihan terbaik yang tersedia pada setiap langkah (locally optimal choice). Strategi ini berprinsip bahwa dengan terus memilih yang terbaik secara lokal, akan diperoleh solusi terbaik secara global.

Greedy diterapkan pada persoalan optimasi yang memiliki dua sifat utama:

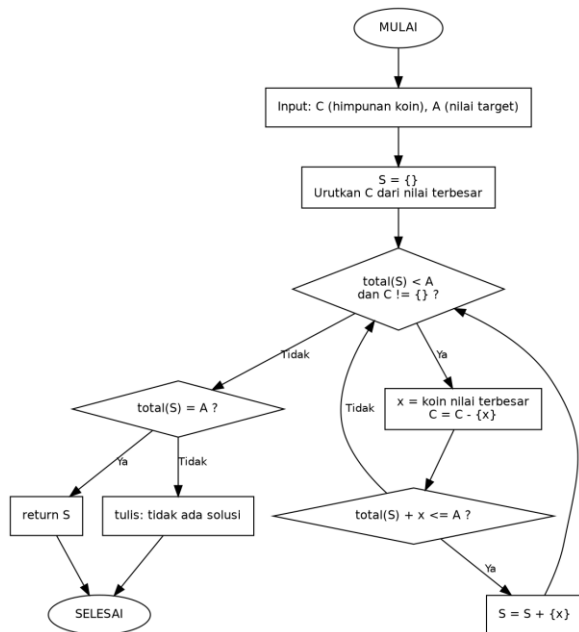
1. **Greedy Choice Property:** solusi global optimal dapat dicapai dengan membuat pilihan lokal yang optimal.
2. **Optimal Substructure:** solusi optimal dari masalah mengandung solusi optimal dari sub-masalahnya.

Contoh persoalan yang dapat diselesaikan dengan Greedy antara lain penukaran uang dengan sistem mata uang kanonik, algoritma Kruskal dan Prim untuk minimum spanning tree, algoritma Dijkstra untuk shortest path, serta Huffman Coding untuk kompresi data. Kelemahannya, Greedy tidak selalu menghasilkan solusi optimal apabila kedua sifat di atas tidak terpenuhi.

## III. ANALISIS PRAKTIKUM

### 3.1 Langkah Praktikum: Coin Exchange

**Flowchart**



Gambar 3.1 Flowchart Algoritma Coin Exchange

### Source Code Python

```

def coin_exchange(coins, amount):
    # urutkan koin dari nilai terbesar ke terkecil (strategi greedy)
    coins_sorted = sorted(coins, reverse=True)
    result = []
    remaining = amount

    for coin in coins_sorted:
        # ambil koin sebanyak mungkin selama tidak melebihi sisa
        while remaining >= coin:
            result.append(coin)
            remaining -= coin

    if remaining == 0:
        return result
    else:
        return None # tidak ada solusi
  
```

### Analisis Output

Program diuji dengan koin standar [1000, 500, 200, 100, 50, 25, 10, 5, 1] dan nilai target 2750:

| Langkah | Koin Dipilih | Sisa Nilai |
|---------|--------------|------------|
| 1       | 1000         | 1750       |
| 2       | 1000         | 750        |
| 3       | 500          | 250        |
| 4       | 200          | 50         |
| 5       | 50           | 0          |

Hasil: koin yang digunakan = [1000, 1000, 500, 200, 50], total 5 koin. Algoritma greedy berhasil menemukan solusi minimal karena sistem koin Rupiah bersifat kanonik, artinya

pilihan koin terbesar di setiap langkah selalu mengarah ke solusi dengan jumlah koin minimum.

### 3.2 Post Test

#### Source Code Python

```
def analisis_kasus(coins, amount, label=""):
    print(f"\n{'='*50}")
    if label:
        print(f"  {label}")
    print(f"{'='*50}")
    print(f"Nilai ditukar : {amount}")
    print(f"Koin tersedia : {sorted(coins, reverse=True)}")

    hasil = coin_exchange(coins, amount)

    if hasil:
        rekap = Counter(hasil)
        print(f"Solusi      : {dict(rekap)}")
        print(f"Jumlah koin   : {len(hasil)}")
    else:
        print("Tidak ada solusi (greedy gagal).")

    return hasil

def hitung_kompleksitas(coins, amount):
    # hitung iterasi aktual untuk analisis kompleksitas
    coins_sorted = sorted(coins, reverse=True)
    n = len(coins_sorted)
    iterasi = 0
    remaining = amount

    for coin in coins_sorted:
        while remaining >= coin:
            remaining -= coin
            iterasi += 1

    print(f"\nAnalisis Kompleksitas (amount={amount}, n={n} jenis koin):")
    print(f"Jumlah iterasi while   : {iterasi}")
    print(f"Jumlah iterasi for     : {n}")
    print(f"T(n) = O(n + amount/koin_terkecil) -> praktis O(n) untuk denominasi wajar")

if __name__ == "__main__":
    koin_std = [1000, 500, 200, 100, 50, 25, 10, 5, 1]

    # kasus 1: solusi optimal
    analisis_kasus(koin_std, 2750, "Kasus 1: Solusi Normal")

    # kasus 2: jumlah besar
    analisis_kasus(koin_std, 9875, "Kasus 2: Jumlah Besar")

    # kasus 3: greedy tidak optimal (denominasi tidak kanonik)
    koin_non_kanonik = [10, 6, 1]
    analisis_kasus(koin_non_kanonik, 12, "Kasus 3: Greedy Tidak Optimal (koin [10,6,1], target 12)")
    print("  Solusi greedy: 10+1+1 = 3 koin")
    print("  Solusi optimal: 6+6   = 2 koin")

    # analisis kompleksitas
    hitung_kompleksitas(koin_std, 2750)
```

#### Jawaban 1: Implementasi, output, dan analisis kasus

Program diuji pada tiga kasus:

- **Kasus 1 (nilai 2750, koin kanonik):** Solusi = {1000:2, 500:1, 200:1, 50:1}, total 5 koin. Greedy optimal.
- **Kasus 2 (nilai 9875, koin kanonik):** Solusi = {1000:9, 500:1, 200:1, 100:1, 50:1, 25:1}, total 14 koin. Greedy optimal.
- **Kasus 3 (nilai 12, koin [10,6,1] non-kanonik):** Solusi greedy = {10:1, 1:2} = 3 koin. Solusi optimal = {6:2} = 2 koin. Greedy tidak optimal karena sistem koin tidak kanonik.

Kasus 3 membuktikan bahwa algoritma Greedy tidak menjamin solusi optimal secara universal. Untuk sistem koin non-kanonik, diperlukan pendekatan Dynamic Programming.

## Jawaban 2: Kompleksitas waktu

Analisis kompleksitas dilakukan berdasarkan dua komponen utama:

- **Loop for iterasi jenis koin:** dilakukan sebanyak  $n$  kali ( $n$  = jumlah jenis koin). Kompleksitas  $O(n)$ .
- **Loop while pengambilan koin:** untuk setiap jenis koin, jumlah pengambilan bergantung pada nilai koin dan sisa amount. Dalam kasus terburuk total iterasi while =  $\text{floor}(A / \text{koin\_terkecil})$ .

Sehingga total kompleksitas:

$$T(n) = O(n + A / d_{\min})$$

dengan  $n$  = jumlah jenis koin dan  $d_{\min}$  = nilai koin terkecil. Untuk sistem mata uang praktis ( $d_{\min} = 1$ ), kasus terburuk adalah  $T(n) = O(A)$ . Namun secara praktis karena koin terbesar mendominasi pengambilan, jumlah iterasi while jauh lebih kecil dari  $A$  sehingga kompleksitas efektif mendekati  $O(n)$ .

| Komponen                           | Kompleksitas                 |
|------------------------------------|------------------------------|
| Pengurutan koin (sorting)          | $O(n \log n)$                |
| Iterasi jenis koin (for)           | $O(n)$                       |
| Pengambilan koin per jenis (while) | $O(A / d_{\min})$ worst case |
| Total keseluruhan                  | $O(n \log n + A / d_{\min})$ |

## IV. IMPLEMENTASI NYATA

Algoritma Greedy pada masalah coin change diterapkan secara nyata dalam sistem transaksi cryptocurrency, khususnya pada proses coin selection pada dompet digital berbasis UTXO (Unspent Transaction Output).

[1] **Penulis:** Wei, X., Wu, C., Yu, H., Luo, X., dan Huang, S. (2023)

**Judul:** *"A Coin Selection Strategy Based on the Greedy and Genetic Algorithm"*

**Publikasi:** Complex and Intelligent Systems, Vol. 9, Hal. 421-434. Springer Nature.  
DOI: 10.1007/s40747-022-00799-2

**Intisari:** Makalah ini membahas penerapan Greedy pada proses coin selection di dompet Bitcoin berbasis UTXO. Metode greedy digunakan sebagai komponen utama untuk memilih set koin yang mendekati nilai target transaksi, kemudian dikombinasikan dengan algoritma genetika untuk meminimalkan biaya transaksi dan mengurangi akumulasi UTXO bernilai kecil (dust). Hasil eksperimen menunjukkan strategi greedy-genetika mengungguli metode greedy murni dalam efisiensi jaringan jangka panjang, meski greedy murni tetap lebih cepat untuk transaksi individual.

**Link:** <https://link.springer.com/article/10.1007/s40747-022-00799-2>

#### **Relevansi dengan Praktikum:**

Seperti halnya penukaran uang konvensional, coin selection pada Bitcoin menggunakan prinsip yang sama yaitu memilih koin dengan nilai terbesar terlebih dahulu untuk meminimalkan jumlah koin yang dipakai. Kompleksitas  $O(n)$  pada greedy murni membuatnya sangat cepat untuk transaksi real-time.

#### **Kelebihan dalam konteks ini:**

- Waktu eksekusi sangat cepat, cocok untuk sistem transaksi real-time yang membutuhkan respons instan.
- Implementasi sederhana dan deterministik sehingga mudah diaudit dan diverifikasi.
- Pada sistem koin kanonik (mata uang resmi maupun Bitcoin), greedy menghasilkan solusi yang optimal atau mendekati optimal.

#### **Kekurangan dalam konteks ini:**

- Tidak optimal untuk sistem koin non-kanonik: greedy bisa menghasilkan lebih banyak koin dari yang seharusnya.
- Tidak mempertimbangkan dampak jangka panjang seperti akumulasi dust UTXO yang memperbesar beban jaringan, sehingga perlu dikombinasikan dengan metode lain seperti algoritma genetika.